

JAWABAN PRAKTIKUM

FAIZA MICHELLE ARFAN - 235150407111031

Percobaan 1: State Holsting

1. Setiap kali tombol + ditekan, jumlah tiket bertambah satu. Setiap kali tombol - ditekan, jumlah tiket berkurang satu selama nilainya lebih dari 1. Total pembayaran juga berubah mengikuti jumlah tiket. Perubahan terjadi karena jumlahTiket merupakan state Compose yang diamati oleh UI.
2. Perubahan terjadi karena jumlahTiket dideklarasikan dengan mutableStateOf dan disimpan menggunakan rememberSaveable. Ketika nilai state berubah, Jetpack Compose mengetahui bahwa UI yang menggunakan state tersebut perlu melakukan recomposition.
3. State yang berperan pada tampilan jumlah tiket adalah variabel jumlahTiket. Nilai tersebut digunakan pada Text jumlah tiket dan juga digunakan dalam perhitungan totalBayar.
4. Tidak terdapat state label tombol seperti pada contoh CounterScreen di modul. Label tombol pada program ini adalah teks statis, yaitu '-' dan '+'. Oleh karena itu, tidak ada variabel state khusus untuk label tombol.
5. Perubahan dari increment 1 menjadi increment 2 pada konteks aplikasi tiket dapat diterapkan dengan mengubah logika callback tombol +, misalnya jumlahTiket + 2. Perubahan dilakukan pada lambda onClick tombol + atau pada fungsi callback yang diteruskan dari TicketScreen. Label tombol juga dapat diubah jika requirement menghendaki tombol menampilkan '+2'.
6. Setelah state di-hoist, perubahan jumlah tiket dilakukan pada TicketScreen melalui callback onJumlahTiketChange. TicketContent hanya menerima jumlahTiket dan menjalankan callback ketika terjadi event klik.
7. Jika kebutuhan berubah, misalnya tombol + harus menambah dua tiket, perubahan utama cukup dilakukan pada logika event di TicketScreen atau pada callback yang diberikan kepada TicketContent. TicketContent tetap dapat digunakan untuk menampilkan jumlah tiket karena tidak bergantung pada bagaimana state tersebut dihitung. Hal ini menunjukkan manfaat state hoisting dalam memisahkan UI dari pengelolaan state dan logika perubahan.

Percobaan 2: Perubahan Konfigurasi

1. Jika aplikasi menggunakan remember dan jumlah tiket sudah diubah dari nilai awal, perubahan konfigurasi seperti rotasi dapat menyebabkan Activity dibuat lagi sehingga state yang hanya disimpan oleh remember tidak terjamin tetap ada, yang menyebabkan jumlah tiket dapat berubah kembali ke nilai awal.
2. Setelah menggunakan rememberSaveable, jumlah tiket tetap dipertahankan ketika perangkat diputar. Misalnya jumlah tiket sebelum rotasi adalah 3, maka setelah rotasi nilai jumlah tiket tetap 3 dan total pembayaran tetap Rp150.000.
3. rememberSaveable berfungsi untuk menyimpan nilai state yang dapat disimpan dalam saved instance state/Bundle sehingga nilai tersebut dapat dipulihkan ketika Activity dibuat ulang karena perubahan konfigurasi. Berbeda dari remember yang berfokus pada mempertahankan state selama recomposition. Namun, rememberSaveable bukan penyimpanan permanen; yang berarti perubahan tidak dipertahankan jika Activity ditutup sepenuhnya oleh pengguna.
4. Contoh penggunaan lain adalah form pendaftaran. Jika pengguna sudah mengisi nama, alamat, atau pilihan checkbox lalu perangkat diputar, rememberSaveable dapat digunakan agar input ringan tersebut tidak langsung kembali ke nilai awal. Manfaatnya adalah pengalaman pengguna

lebih baik karena data input dapat dipulihkan setelah perubahan konfigurasi. Modul juga menyebut input teks, status checkbox, dan posisi scroll sebagai contoh data ringan yang sesuai.

Percobaan 3: Snackbar

1. Saat tombol + atau - ditekan, Snackbar menampilkan pesan seperti 'Jumlah tiket: 2'. Jika belum ada perubahan jumlah tiket, tidak ada Snackbar yang dipicu oleh callback tersebut.
2. Snackbar muncul karena `scope.launch` menjalankan `snackbarHostState.showSnackbar` setelah `jumlahTiket` diubah. Pada kode ini pemicunya adalah event klik tombol yang menjalankan `onJumlahTiketChange`.
3. Saat tombol + ditekan satu kali, jumlah tiket bertambah satu, total pembayaran berubah, dan Snackbar menampilkan jumlah tiket terbaru. Hal tersebut terjadi karena state `jumlahTiket` diperbarui dan callback kemudian menjalankan side-effect `showSnackbar`.
4. Snackbar dapat dipicu berdasarkan perubahan state `jumlahTiket`. Oleh karena itu, perubahan state `jumlahTiket` menjadi kunci yang menyebabkan effect dijalankan kembali.
5. `LaunchedEffect(state)` menggunakan state sebagai key. Effect dijalankan ketika composable masuk komposisi dan dijalankan kembali ketika nilai key berubah; effect lama dibatalkan ketika key berubah. `LaunchedEffect(Unit)` menggunakan key yang tetap sehingga effect pada dasarnya dijalankan ketika composable pertama kali masuk komposisi dan tidak dipicu ulang hanya karena state lain berubah. Modul menggunakan `LaunchedEffect(Unit)` untuk simulasi pengambilan data selama dua detik.